LAB # 01

Introduction to Deep Learning & TensorFlow Fundamentals

Lab Tasks:

- Create a random float Tensor and compute sum, mean, maximum.

Code:-

```
import tensorflow as tf

# Create a random float tensor (shape: 3x3)
tensor = tf.random.uniform(shape=(3, 3), minval=0, maxval=1, dtype=tf.float32)

print("Tensor:\n", tensor.numpy())

# Compute sum, mean, and maximum
tensor_sum = tf.reduce_sum(tensor)
tensor_mean = tf.reduce_mean(tensor)
tensor_max = tf.reduce_max(tensor)

print("\nSum:", tensor_sum.numpy())
print("Mean:", tensor_mean.numpy())
print("Maximum:", tensor_max.numpy())
```

Output:-

```
Tensor:
 [[0.04096818 0.8523736  0.43541253]
 [0.61958075 0.18590891 0.5329983 ]
 [0.8218131  0.20435846 0.941692  ]]

Sum: 4.6351056
Mean: 0.5150117
Maximum: 0.941692
```

- Create a 3D TensorFlow tensor and access a specific element, extract a 2D slice, and extract a 1D slice.

Code:-

```
import tensorflow as tf
tensor_3d = tf.constant([
    [[1, 2, 3, 4],
     [5, 6, 7, 8],
     [9, 10, 11, 12]],

    [[13, 14, 15, 16],
     [17, 18, 19, 20],
     [21, 22, 23, 24]]
])

print("3D Tensor:\n", tensor_3d.numpy())

element = tensor_3d[1, 2, 3]
print("\nSpecific element [1,2,3]:", element.numpy())

slice_2d = tensor_3d[0]
print("\n2D Slice (tensor_3d[0]):\n", slice_2d.numpy())

slice_1d = tensor_3d[0, 1]
print("\n1D Slice (tensor_3d[0,1]):\n", slice_1d.numpy())
```

Output:-

```
3D Tensor:
 [[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[13 14 15 16]
  [17 18 19 20]
  [21 22 23 24]]]

Specific element [1,2,3]: 24
```

- Create a NumPy array with random float values. Sort the array in ascending and descending order.

Code:-

```
import numpy as np

# Create a random float array (size: 10)
arr = np.random.rand(10)

print("Original Array:\n", arr)

# Sort in ascending order
asc_sorted = np.sort(arr)
print("\nAscending Order:\n", asc_sorted)

# Sort in descending order
desc_sorted = np.sort(arr)[::-1]
print("\nDescending Order:\n", desc_sorted)
```

Output:-

```
Original Array:
 [0.04459236 0.57812315 0.71483384 0.86587718 0.7441134  0.4856163
 0.27428968 0.03509996 0.15517855 0.40364854]

Ascending Order:
 [0.03509996 0.04459236 0.15517855 0.27428968 0.40364854 0.4856163
 0.57812315 0.71483384 0.7441134  0.86587718]

Descending Order:
 [0.86587718 0.7441134  0.71483384 0.57812315 0.4856163  0.40364854
 0.27428968 0.15517855 0.04459236 0.03509996]
```

- Create a 2x2 NumPy matrix. Calculate its transpose and inverse.

Code:-

```
import numpy as np

# Create a 2x2 matrix
matrix = np.array([[4, 7],
                   [2, 6]])

print("Original Matrix:\n", matrix)

# Transpose of the matrix
transpose = matrix.T
print("\nTranspose:\n", transpose)

# Inverse of the matrix
inverse = np.linalg.inv(matrix)
print("\nInverse:\n", inverse)
```

Output:-

```
Original Matrix:
 [[4 7]
 [2 6]]

Transpose:
 [[4 2]
 [7 6]]

Inverse:
 [[ 0.6 -0.7]
 [-0.2  0.4]]
```

- Create a 2D NumPy array. Calculate the sum of each row, the mean of each column, and the standard deviation of the entire array

Code:-

```
import numpy as np

# Create a 2D array (3x3)
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

print("Array:\n", arr)

#Sum of each row
row_sum = np.sum(arr, axis=1)
print("\nSum of each row:", row_sum)

#Mean of each column
col_mean = np.mean(arr, axis=0)
print("Mean of each column:", col_mean)

#Standard deviation of entire array
std_dev = np.std(arr)
print("Standard deviation of array:", std_dev)
```

Output:-

```
Array:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]

Sum of each row: [ 6 15 24]
Mean of each column: [4. 5. 6.]
Standard deviation of array: 2.581988897471611
```

Bonus Task:

Convert a NumPy array to a TensorFlow tensor.

Perform tensor operations (addition, multiplication) on this tensor.

Design a function that:

- Accepts a NumPy array as input
- Converts it into a TensorFlow tensor
- Adds 10 to each element
- Returns the updated tensor

Code:-

```
import numpy as np
import tensorflow as tf

# Create a NumPy array
np_array = np.array([1, 2, 3, 4, 5])

# Convert to TensorFlow tensor
tensor = tf.convert_to_tensor(np_array)

print("Tensor:", tensor)

# Addition (add 5 to each element)
add_result = tensor + 5
print("\nAfter Addition:", add_result.numpy())

# Multiplication (multiply by 2)
mul_result = tensor * 2
print("\nAfter Multiplication:", mul_result.numpy())
```

Output:-

```
Tensor: tf.Tensor([1 2 3 4 5], shape=(5,), dtype=int64)

After Addition: [ 6  7  8  9 10]

After Multiplication: [ 2  4  6  8 10]
```

```
def process_array(np_array):
    import tensorflow as tf

    # Convert NumPy array to TensorFlow tensor
    tensor = tf.convert_to_tensor(np_array)

    # Add 10 to each element
    updated_tensor = tensor + 10

    return updated_tensor

import numpy as np
arr = np.array([10, 20, 30])

result = process_array(arr)
print("\nUpdated Tensor:", result.numpy())
```

```
Updated Tensor: [20 30 40]
```